

Lang Graph Agent – Coding Task (Invoice Processing Workflow)

Task Overview

Use **vibe coding** — leverage AI coding assistants (Cursor, Claude, GitHub Copilot, etc.) to write most of the code. You must **review, test, and validate** AI-generated code so it works end-to-end. We highly encourage AI-assisted development.

You are required to design and implement a **Lang Graph Agent** that models an **Invoice Processing workflow** as a graph of sequential stages.

Each stage represents a clearly defined step in the workflow, and the agent must **persist and pass state variables across stages**, including storing data at **checkpoint states** and resuming execution after **Human-In-The-Loop (HITL)** decisions.

The agent must support:

- **Deterministic stages** (executed one after another)
- **Non-deterministic stages** (dynamic logic based on context)
- **HITL Checkpoints** (pause workflow → human review → resume)
- **Integration with MCP Clients** (to access ATLAS or COMMON servers for ability execution)
- **Bigtool** (to dynamically select tools from a tool pool for OCR, Enrichment, ERP access, etc.)

The goal is to implement a full **invoice-processing workflow** using Lang Graph, mapping multiple stages with abilities, and demonstrating:

- ➡ reasoning,
 - ➡ state management,
 - ➡ checkpoint & resume,
 - ➡ HITL routing,
 - ➡ Bigtool-based tool selection,
 - ➡ MCP client orchestration.
-

Agent Capabilities

Graph Orchestration Agent (Lang Graph)

Your agent must:

- Represent each workflow stage as a **node** with persistent state
- Execute **deterministic nodes sequentially**
- Execute **non-deterministic nodes** by choosing abilities at runtime
- Route abilities through MCP clients to:
 - **COMMON Server** → abilities requiring no external data
 - **ATLAS Server** → abilities requiring external system interaction (ERP, enrichment services)
- Support **checkpoint-based execution** using LangGraph's native **checkpoints** feature
- Store checkpoint state in DB so it appears under the **Human Review tab** in the app
- Resume workflow execution from the checkpoint after human action

Invoice Processing Agent Flow (New Workflow)

Implement the following stages as LangGraph nodes:

-
1. INTAKE  – `accept_invoice_payload` (Deterministic)

- Accept invoice payload (raw data + attachments)
- Validate schema
- Persist raw invoice

Servers: COMMON

2. UNDERSTAND – ocr_extract, parse_line_items (Deterministic)

- Run OCR on invoice attachments
- Parse line items, amounts, PO references

Servers:

- OCR via ATLAS (external)
- Parsing via COMMON

Bigtool must select OCR provider (Google Vision / Tesseract / AWS Textract).

3. PREPARE – normalize_vendor, enrich_vendor, compute_flags (Deterministic)

- Normalize vendor name
- Enrich vendor data (PAN/GST/TaxID), credit score, risk score
- Compute validation flags

Servers:

- COMMON: normalize_vendor, compute_flags
- ATLAS: enrich_vendor

Bigtool chooses the enrichment tool (Clearbit / PDL / Vendor DB).

4. RETRIEVE – fetch_po, fetch_grn, fetch_history (Deterministic)

- Fetch Purchase Orders
- Fetch Goods Received Notes
- Fetch historical invoices

Servers: ATLAS (ERP connector)

Bigtool chooses ERP connector tool.

5. MATCH_TWO_WAY – compute_match_score (Deterministic)

- Perform **2-way matching**: Invoice vs PO
- Compute match_score (0–1)
- If **match_score < threshold** → mark for **HITL CHECKPOINT**

Servers: COMMON

6. CHECKPOINT_HITL – save_state_for_human_review (Deterministic)

Triggered **ONLY IF** matching fails.

This stage must:

- Create a **LangGraph Checkpoint**
- Persist full workflow state to DB
- Add entry into **Human Review queue**

- Generate a `review_url` for the reviewer
- Pause execution (`PAUSED` state)

Servers: COMMON

Bigtool selects DB tool (Postgres / SQLite / Dynamo etc.)

7. HITL_DECISION – accept_or_reject_invoice (Non-Deterministic)

Executed when a human acts on the review UI.

Your job:

- Read stored checkpoint state
- Get reviewer decision (`ACCEPT` / `REJECT`)
- If **ACCEPT** → resume workflow at the next node (`RECONCILE`)
- If **REJECT** → finalize workflow with status `REQUIRES_MANUAL_HANDLING`

Servers: ATLAS

8. RECONCILE – build_accounting_entries (Deterministic)

- Reconstruct accounting entries
- Build payable/receivable ledger entries

Servers: COMMON

9. APPROVE – apply_invoice_approval_policy (Deterministic)

- Auto-approve or escalate based on invoice amount + rules

Servers: ATLAS (if integration needed)

10. POSTING 🏃 – post_to_erp, schedule_payment (Deterministic)

- Post entries to ERP/AP
- Schedule payment

Servers: ATLAS

11. NOTIFY ✉️ – notify_vendor, notify_finance_team (Deterministic)

- Notify vendor and internal team

Servers: ATLAS

12. COMPLETE ✅ – output_final_payload (Deterministic)

- Produce final structured payload
- Output logs
- Mark workflow complete

Servers: COMMON



Prompt Template (Agent Personality)

You are **Langie – the Invoice Processing LangGraph Agent**.

You think in structured stages.

Each node is a well-defined processing phase.

You always carry forward **state variables** between nodes.

You know when to execute deterministic steps and when to choose dynamically.
You orchestrate MCP clients to call **COMMON** or **ATLAS** abilities as required.
You use **Bigtool** whenever a tool must be selected from a pool.
You log every decision, every tool choice, and every state update.
You always produce clean structured output.

Expected Deliverables

1. LangGraph Agent Config (JSON)

Must include:

- Input workflow.json schema: Check sample in APPENDIX-1.
 - Stages definition with mode: deterministic / non-deterministic
 - Ability → MCP server mapping (COMMON or ATLAS)
 - Bigtool configuration for choosing OCR/Enrichment/ERP tools
 - HITL logic: checkpoint creation, pause, resume
-

2. Working LangGraph Implementation

Your implementation must:

- Build lang graph based on the workflow.json input.
- Execute all stages in order
- Persist and pass state across nodes
- Create & store checkpoints on matching failure
- Expose an API for human review (accept/reject)
- Resume workflow after HITL decision

- Show Bigtool tool selections per stage
 - Integrate with MCP Clients for ability execution
-

3. Demo Run

Provide:

- **Input:** Sample invoice JSON
 - **Output:** Final structured payload after all stages
 - **Logs:**
 - Stage-by-stage execution
 - Ability calls
 - Bigtool selections
 - Checkpoint creation
 - Human resume event
-

Recommended Steps

- **Stage Modeling:** Define all 12 stages in LangGraph
- **State Management:** Persist state across stages + DB checkpoints
- **Checkpointing:** Integrate LangGraph checkpoint store + DB
- **HITL Flow:** Pause workflow → store for human review → resume
- **MCP Integration:** Route abilities to COMMON/ATLAS servers

- **Bigtool Integration:** Implement tool selection
- **Validation:** Run full workflow and show logs



Method of Submission

Submission Checklist

- GitHub repo with full LangGraph implementation + config
- Latest resume attached
- Must have **Demo video** link on onedrive or google drive etc:
 - Self introduction -1 min
 - Demo of working solution from frontend UI app, execution logs etc (**must speak and explain in English**). -4 min

Send to:

santosh.thota@analytos.ai

Cc:

shashwat.shlok@analytos.ai

gaurav.gupta@analytos.ai

Subject:

LangGraph Invoice Processing Task with HITL – <Your Name>



ALL THE BEST!

We're excited to see how you leverage **LangGraph + MCP + HITL + Bigtool** to build a truly autonomous, resilient, human-guided invoice processing agent.



Appendix-1

```
{
  "version": "1.0",
  "workflow_name": "InvoiceProcessing_v1",
  "description": "LangGraph invoice processing with HITL checkpoint/resume and Bigtool tool selection.",
  "config": {
    "match_threshold": 0.90,
    "two_way_tolerance_pct": 5,
    "human_review_queue": "human_review_queue",
    "checkpoint_table": "checkpoints",
    "default_db": "sqlite:///./demo.db"
  },
  "inputs": {
    "invoice_payload": {
      "invoice_id": "string",
      "vendor_name": "string",
      "vendor_tax_id": "string",
      "invoice_date": "string",
      "due_date": "string",
      "amount": "number",
      "currency": "string",
      "line_items": [
        { "desc": "string", "qty": "number", "unit_price": "number", "total": "number" }
      ],
      "attachments": ["string"]
    }
  },
  "stages": [
```

```

{
  "id": "INTAKE",
  "mode": "deterministic",
  "agent": "IngestNode",
  "instructions": "Validate payload schema, persist raw invoice payload and attachments metadata. Return raw_id
and ingest timestamp.",
  "tools": [
    { "name": "BigtoolPicker", "capability": "storage", "action": "select", "pool_hint": ["s3", "gcs", "local_fs"] },
    { "name": "DB", "config_ref": "{{DB_CONN}}" }
  ],
  "output_schema": {
    "raw_id": "string",
    "ingest_ts": "string",
    "validated": "boolean"
  }
},
{
  "id": "UNDERSTAND",
  "mode": "deterministic",
  "agent": "OcrNlpNode",
  "instructions": "Run OCR on attachments, extract text and parse line items, normalize dates/currency, return
parsed_invoice.",
  "tools": [
    { "name": "BigtoolPicker", "capability": "ocr", "action": "select", "pool_hint":
["google_vision", "tesseract", "aws_textract"] },
    { "name": "NLPParser", "config_ref": "{{NLP_KEY}}" }
  ],
  "output_schema": {
    "parsed_invoice": {
      "invoice_text": "string",
      "parsed_line_items": "array",
      "detected_pos": "array",

```

```

    "currency": "string",

    "parsed_dates": { "invoice_date": "string", "due_date": "string" }

  }

},

{

  "id": "PREPARE",

  "mode": "deterministic",

  "agent": "NormalizeEnrichNode",

  "instructions": "Normalize vendor name, enrich vendor profile and compute flags (risk, missing_info). Use Bigtool
to pick enrichment provider.",

  "tools": [

    { "name": "BigtoolPicker", "capability": "enrichment", "action": "select", "pool_hint":
["clearbit","people_data_labs","vendor_db"] },

    { "name": "COMMON_utils", "config_ref": "{{COMMON_KEY}}" }

  ],

  "output_schema": {

    "vendor_profile": { "normalized_name": "string", "tax_id": "string", "enrichment_meta": "object" },

    "normalized_invoice": { "amount": "number", "currency": "string", "line_items": "array" },

    "flags": { "missing_info": "array", "risk_score": "number" }

  }

},

{

  "id": "RETRIEVE",

  "mode": "deterministic",

  "agent": "ErpFetchNode",

  "instructions": "Fetch POs, GRNs and historical invoices from ERP/Procurement systems to find candidate
matches.",

  "tools": [

    { "name": "BigtoolPicker", "capability": "erp_connector", "action": "select", "pool_hint":
["sap_sandbox","netsuite","mock_erp"] },

    { "name": "ATLAS_client", "config_ref": "{{ATLAS_ERP_KEY}}" }

  ],

```

```

"output_schema": {
  "matched_pos": "array",
  "matched_grns": "array",
  "history": "array"
}
},
{
  "id": "MATCH_TWO_WAY",
  "mode": "deterministic",
  "agent": "TwoWayMatcherNode",
  "instructions": "Compute 2-way match score between invoice and PO. If match_score >= config.match_threshold
set match_result='MATCHED' else 'FAILED'. Include tolerance analysis.",
  "tools": [
    { "name": "MatchEngine", "config_ref": "{{MATCH_KEY}}" },
    { "name": "COMMON_utils", "config_ref": "{{COMMON_KEY}}" }
  ],
  "output_schema": {
    "match_score": "number",
    "match_result": "string",
    "tolerance_pct": "number",
    "match_evidence": "object"
  }
},
{
  "id": "CHECKPOINT_HITL",
  "mode": "deterministic",
  "agent": "CheckpointNode",
  "instructions": "If match_result == 'FAILED' persist full state as a checkpoint (state_blob) in DB, create review
ticket and push to human review queue. Return checkpoint_id and review_url. Pause workflow.",
  "trigger_condition": "input_state.match_result == 'FAILED'",
  "tools": [
    { "name": "BigtoolPicker", "capability": "db", "action": "select", "pool_hint": ["postgres", "sqlite", "dynamodb"] },

```

```

    { "name": "QueueService", "config_ref": "{{QUEUE_KEY}}" }
  ],
  "output_schema": {
    "checkpoint_id": "string",
    "review_url": "string",
    "paused_reason": "string"
  }
},
{
  "id": "HITL_DECISION",
  "mode": "non-deterministic",
  "agent": "HumanReviewNode",
  "instructions": "Await human decision via human-review API. Accept or Reject. On ACCEPT return resume_token and next_stage='RECONCILE'. On REJECT finalize with status 'MANUAL_HANDOFF'.",
  "tools": [
    { "name": "HumanUI", "config_ref": "{{APP_URL}}" },
    { "name": "Auth", "config_ref": "{{AUTH_KEY}}" }
  ],
  "output_schema": {
    "human_decision": "string",
    "reviewer_id": "string",
    "resume_token": "string",
    "next_stage": "string"
  }
},
{
  "id": "RECONCILE",
  "mode": "deterministic",
  "agent": "ReconciliationNode",
  "instructions": "If human accepted or invoice matched, create accounting entries (debits/credits) and reconciliation report.",
  "tools": [

```

```

    { "name": "AccountingEngine", "config_ref": "{{ACCT_KEY}}" },
    { "name": "COMMON_utils", "config_ref": "{{COMMON_KEY}}" }
],
"output_schema": {
    "accounting_entries": "array",
    "reconciliation_report": "object"
}
},
{
    "id": "APPROVE",
    "mode": "deterministic",
    "agent": "ApprovalNode",
    "instructions": "Apply approval policies (auto-approve under threshold, escalate above). Return approval_status
and approver_id if escalated.",
    "tools": [
        { "name": "WorkflowEngine", "config_ref": "{{WF_KEY}}" }
    ],
    "output_schema": {
        "approval_status": "string",
        "approver_id": "string"
    }
},
{
    "id": "POSTING",
    "mode": "deterministic",
    "agent": "PostingNode",
    "instructions": "Post journal entries to ERP and schedule payment. Return posted flag and txn ids.",
    "tools": [
        { "name": "BigtoolPicker", "capability": "erp_connector", "action": "select", "pool_hint":
["sap_sandbox","netsuite","mock_erp"] },
        { "name": "Payments", "config_ref": "{{PAY_KEY}}" }
    ],

```

```

"output_schema": {
  "posted": "boolean",
  "erp_txn_id": "string",
  "scheduled_payment_id": "string"
},
{
  "id": "NOTIFY",
  "mode": "deterministic",
  "agent": "NotifyNode",
  "instructions": "Send notifications to vendor and internal finance team (email/slack). Log notification statuses.",
  "tools": [
    { "name": "BigtoolPicker", "capability": "email", "action": "select", "pool_hint": ["sendgrid", "smartlead", "ses"] },
    { "name": "Messaging", "config_ref": "{{SLACK_KEY}}" }
  ],
  "output_schema": {
    "notify_status": "object",
    "notified_parties": "array"
  }
},
{
  "id": "COMPLETE",
  "mode": "deterministic",
  "agent": "CompleteNode",
  "instructions": "Produce final payload and audit log entries. Mark workflow completed and persist audit to DB.",
  "tools": [
    { "name": "BigtoolPicker", "capability": "db", "action": "select", "pool_hint": ["postgres", "sqlite", "dynamodb"] }
  ],
  "output_schema": {
    "final_payload": "object",
    "audit_log": "array",

```



```
    "status": "string"
  }
}
],
"error_handling": {
  "retry_policy": { "max_retries": 3, "backoff_seconds": 2 },
  "on_unrecoverable_error": { "action": "persist_and_fail", "notify": ["ops_team"] }
},
"human_review_api_contract": {
  "list_pending_endpoint": {
    "path": "/human-review/pending",
    "method": "GET",
    "response_schema": {
      "items": [
        {
          "checkpoint_id": "string",
          "invoice_id": "string",
          "vendor_name": "string",
          "amount": "number",
          "created_at": "string",
          "reason_for_hold": "string",
          "review_url": "string"
        }
      ]
    }
  }
},
"decision_endpoint": {
  "path": "/human-review/decision",
  "method": "POST",
  "request_schema": {
    "checkpoint_id": "string",
```

```

    "decision": "string",

    "notes": "string",

    "reviewer_id": "string"

  },

  "response_schema": {

    "resume_token": "string",

    "next_stage": "string"

  }

},

"tools_hint": {

  "bigtool_picker": "Use BigtoolPicker.select(capability, context) to choose tool implementation. Pool configured via tools.yaml or env.",

  "example_pools": {

    "ocr": ["google_vision", "tesseract", "aws_textract"],

    "enrichment": ["clearbit", "people_data_labs", "vendor_db"],

    "erp_connector": ["sap_sandbox", "netsuite", "mock_erp"],

    "db": ["postgres", "sqlite", "dynamodb"],

    "email": ["sendgrid", "smartlead", "ses"]

  }

},

"notes": "Adjust config refs ({{...}}) to actual env var names. CheckpointNode will set `paused_reason` and produce `review_url`. HumanReviewNode must return a `resume_token` that LangGraph uses to resume execution."

}

```